



University of Idaho

Department of Computer Science

CS 4727/5727
Adversarial
Machine Learning

Dr. Alex Vakanski



Lecture 6

Introduction to Large Language Models



Lecture Content

- 6.1 Introduction to NLP
 - 6.1.1 Word Representation in NLP
 - 6.1.2 Word Embeddings
- 6.2 Transformer Networks
- 6.3 Large Language Models
 - 6.3.1 Creating LLMs
 - 6.3.2 Retrieval Augmented Generation (RAG)
 - 6.3.3 Mixture of Experts
- 6.4 Limitations of LLMs
- 6.5 Reasoning Models
- 6.6 Agentic AI

NLP Tasks

6.1 Introduction to NLP

- Main NLP (Natural Language Processing) tasks include:
 - *Text classification*—assign a class label to text based on the topic discussed in the text
 - E.g., sentiment analysis (positive or negative movie review), spam detection, content filtering (detect abusive content)
 - *Text summarization/reading comprehension*—summarize a long input document with a shorter text
 - *Speech recognition*—convert spoken language to text
 - *Machine translation*—convert text in a source language to a target language
 - *Part of Speech (PoS) tagging*—mark up words in text as nouns, verbs, adverbs, etc.
 - *Question answering*—output an answer to an input question
 - *Dialog generation*—generate the next reply in a conversation given the history of the conversation
 - *Text generation*—generate text to complete the sentence or to complete the paragraph



Text Processing Models

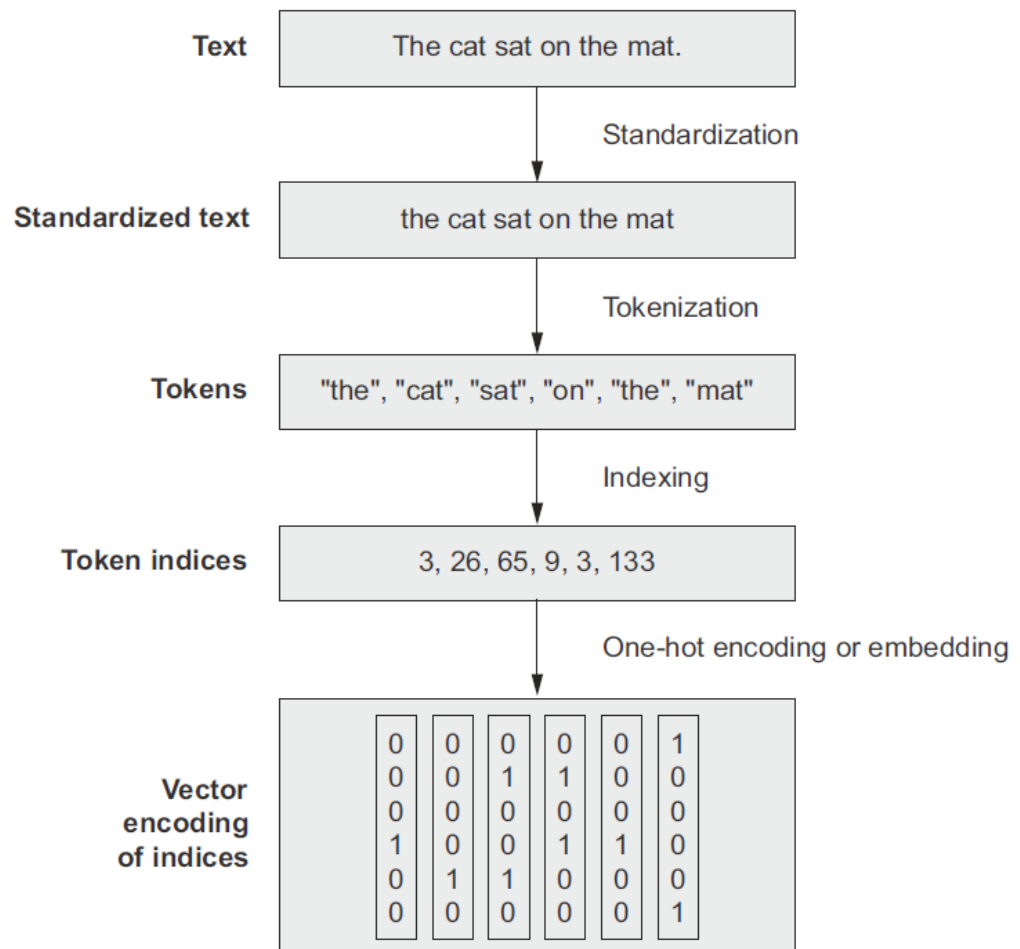
6.1 Introduction to NLP

- Dominant text processing models
 - Pre1980
 - Hand-crafted rule-based approaches (if-then-else rules)
 - 1980 - 2000
 - Statistical Language Models: N -grams, bag-of-words
 - 2000 - 2014
 - Traditional ML models, e.g., decision trees, logistic regression, Naïve Bayes
 - 2014 - 2017
 - Recurrent NNs (e.g., LSTM, GRU) layers
 - Combinations of CNNs and RNNs
 - 2017 - 2022
 - Transformers (Language Models: BERT, GPT 1-2, DistilBERT)
 - 2022 - present
 - Transformers (Large Language Models: GPT 3+, Gemini, Claude, LLaMA)

Preprocessing Text Data

6.1 Introduction to NLP

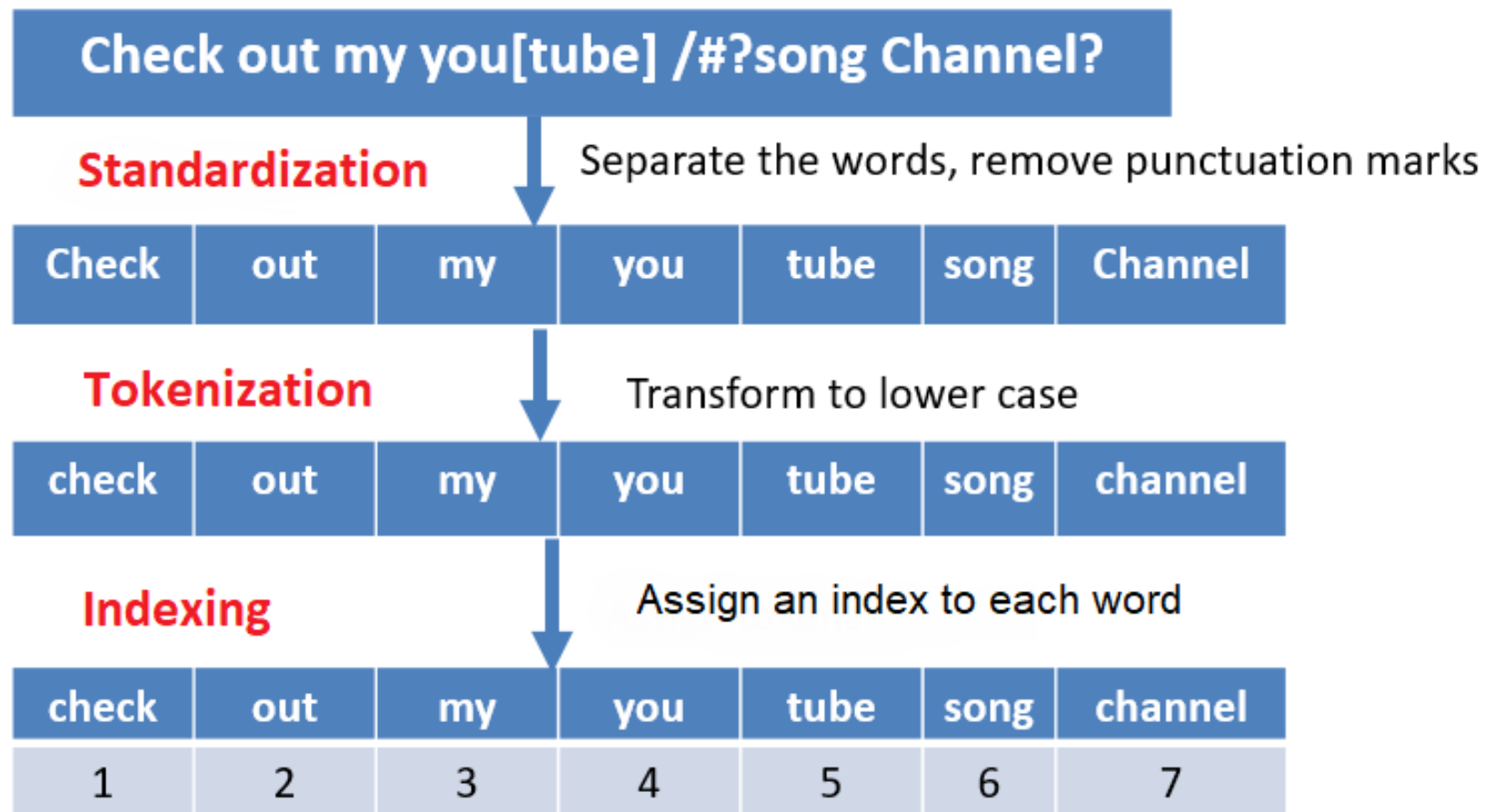
- Converting text data into numerical form for processing by ML models typically involves the following steps
 - **Standardization**
 - Convert to lower case, remove punctuation, lemmatization
 - **Tokenization**
 - Break up the text into tokens
 - Tokens can be individual characters, sub-words, words, or consecutive words (n -grams)
 - **Indexing**
 - Assign a numerical index to each token in the training set (vocabulary)
 - **Vector Encoding**
 - Assign a numerical vector to each index: one-hot encoding or word embedding



Preprocessing Text Data

6.1 Introduction to NLP

- Example of preprocessing text data



Tokenization

6.1.1 Word Representation in NLP

- **Tokenization** can be performed at different levels
 - **Character-level tokenization** - each individual character is a token, including letters, digits, punctuation marks, and symbols
 - Character-level tokenization does not capture semantic meaning of words as effectively as word-level tokens, and it is not widely used in practice
 - E.g., antigrams (words with same letters in different order, such as 'silent' and 'listen') can have the same numerical encoding, which can affect the performance of ML models
 - **Word-level tokenization** - each word is a token
 - Provides a natural representation of text with the words as building blocks of language
 - **Subword-level tokenization** - the words are divided into smaller units
 - E.g., tokenizing the word "unhappiness" into two tokens "un" + "happiness"
 - Word-level and subword-level tokenization are most used at present
 - **n -gram tokenization** - n consecutive words represent a token
- For some NLP tasks, tokenization can also be performed at other levels, such as sentence-level tokenization for document segmentation task



Tokenization

6.1.1 Word Representation in NLP

- Character-level tokenization example
 - Example text: TensorFlow is a Machine Learning framework
 - Tokens are assigned to each character (the first token is the empty space character: ' ': 1, 'e': 2, 'n': 3, 'r': 4, 'a': 5, 'o': 6, 'i': 7, 's': 8, 'f': 9, 'l': 10, 'w': 11, 'm': 12, 't': 13, 'c': 14, 'h': 15, 'g': 16, 'k': 17)
 - Tokenized text: 13, 2, 3, 8, 6, 4, 9, 10, 6, 11, 1, 7, 8, 1, 5, 1, 12, 5, 14, 15, 7, 3, 2, 1, 10, 2, 5, 4, 3, 7, 3, 16, 1, 9, 4, 5, 12, 2, 11, 6, 4, 17
- Word-level tokenization example
 - Example text: TensorFlow is a Machine Learning framework. Keras is a well designed deep learning API! Keras is built on top of TensorFlow!
 - Tokens are assigned to each word: 'is': 1, 'tensorflow': 2, 'a': 3, 'learning': 4, 'keras': 5, 'machine': 6, 'framework': 7, 'well': 8, 'designed': 9, 'deep': 10, 'api': 11, 'built': 12, 'on': 13, 'top': 14, 'of': 15
 - Tokenized text: [2, 1, 3, 6, 4, 7], [5, 1, 3, 8, 9, 10, 4, 11], [5, 1, 12, 13, 14, 15, 2]



n -Grams

6.1.1 Word Representation in NLP

- Instead of using single words or subwords as tokens, it is also possible to use n consecutive words as tokens, referred to as *n -grams*
 - Combining several consecutive words together can create more specialized tokens
 - This type of tokenization is still popular for some NLP tasks
 - E.g., the word *play* is considered a neutral word in spam filtering, but the two-words phrase *play lotto* is less neutral
 - Such n -grams consisting of two adjacent pairs of words are called **bigrams**
 - n -grams consisting of single words are called **unigrams**

Representation of Groups of Words

6.1.1 Word Representation in NLP

- The representation of groups of words in text data can be divided into two categories of approaches:
 - *Set models* approach, where the text is represented as unordered collection of words
 - The order of the words in the text is not preserved
 - Representative technique of this group is the **bag-of-words** model
 - *Sequence models* approach, where the text is represented as ordered sequences of words
 - These methods preserve the order of the words in the text
 - Representatives of this group are **Recurrent Neural Networks** and **Transformer Networks**
- In general, the order of words in natural language is not necessarily fixed, and sentences with different orders of the words can have the same meaning
 - However, in many cases the word order can be very important and difference in the word order can significantly change the meaning of the text
 - Recent ML models for NLP predominantly rely on sequence models where the order of the words is preserved

Bag-of-Words Approach

6.1.1 Word Representation in NLP

• Bag-of-words approach

- The tokenized words in text are represented as a bag (i.e., set) of words
- The term *bag* implies that the order of the words and the structure of the text is lost
 - A numerical value is assigned to each token (can be either individual words or n -grams)
 - The frequency of occurrence of each word is typically used as a feature for training a ML classifier

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1

Sequence Model Approach

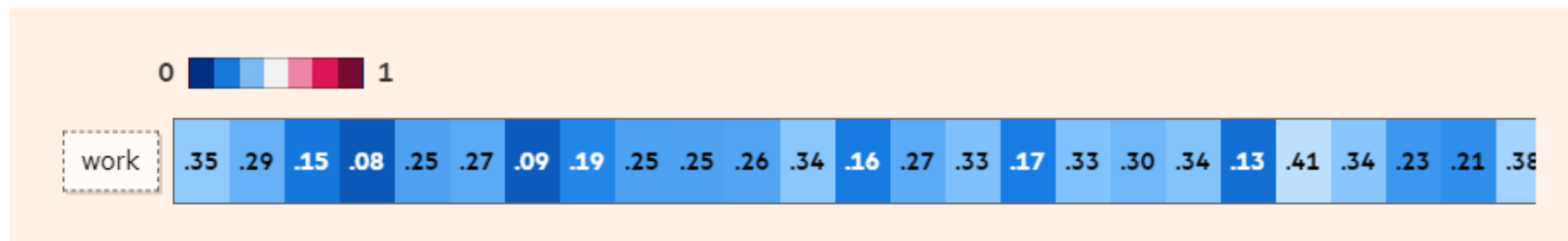
6.1.1 Word Representation in NLP

- *Sequence models* preserve the order of words in the input text
- As mentioned, commonly used models are Recurrent Neural Networks and Transformer Networks
 - Transformers have replaced RNNs in recent applications
- The application of sequence models typically involves:
 1. Tokenization, to represent the words in text data with integer indices
 2. Mapping the integers to vector representations (embeddings)
 3. Pad or truncate the sequences in the text to have the same length
 4. Use the tokenized sequences as inputs to train a machine learning model
- The trained models take into account the ordering of words embeddings in the original text

Word Embeddings

6.1.2 Word Embeddings

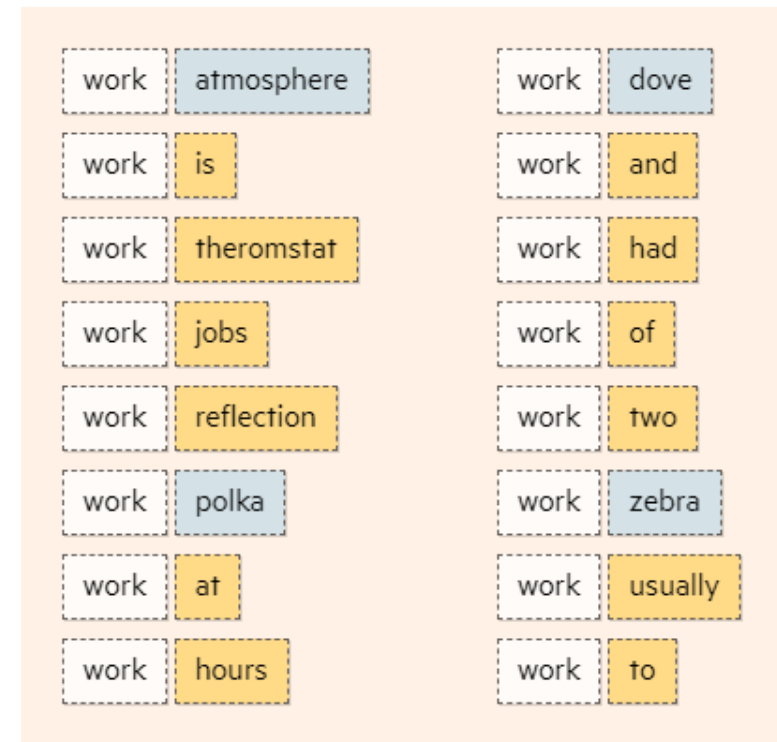
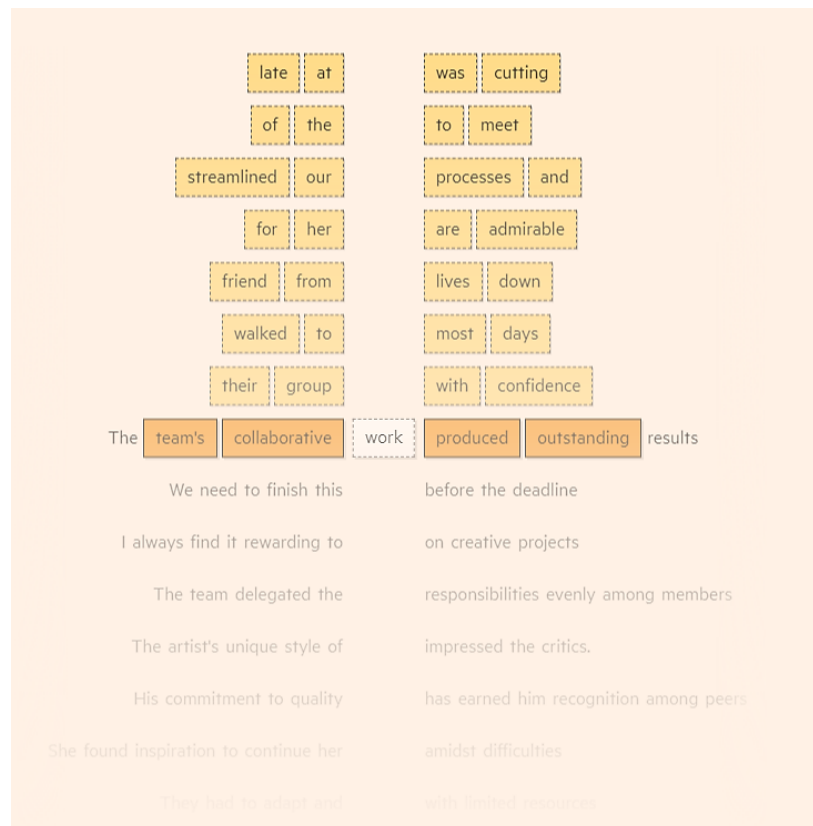
- **Word embedding** is converting words to a vector format, where the vectors represent the position of words in a vector (embedding) space
 - Words that have similar meanings should have close spatial positions of their vector representations in the **embedding space**
- Typical vectors for representing word embeddings have between 256 to 2,048 dimensions
 - E.g., the embedding size in recent open-source LLMs is 2,048 (LlaMa 3.2), 2,560 (Qwen 3 4B), 4,096 (Qwen 3 235B), 7,168 (DeepSeek V3)
- Example: embedding vector for the word 'work'
 - The values represent the elements of the embedding vector



Word Embeddings

6.1.2 Word Embeddings

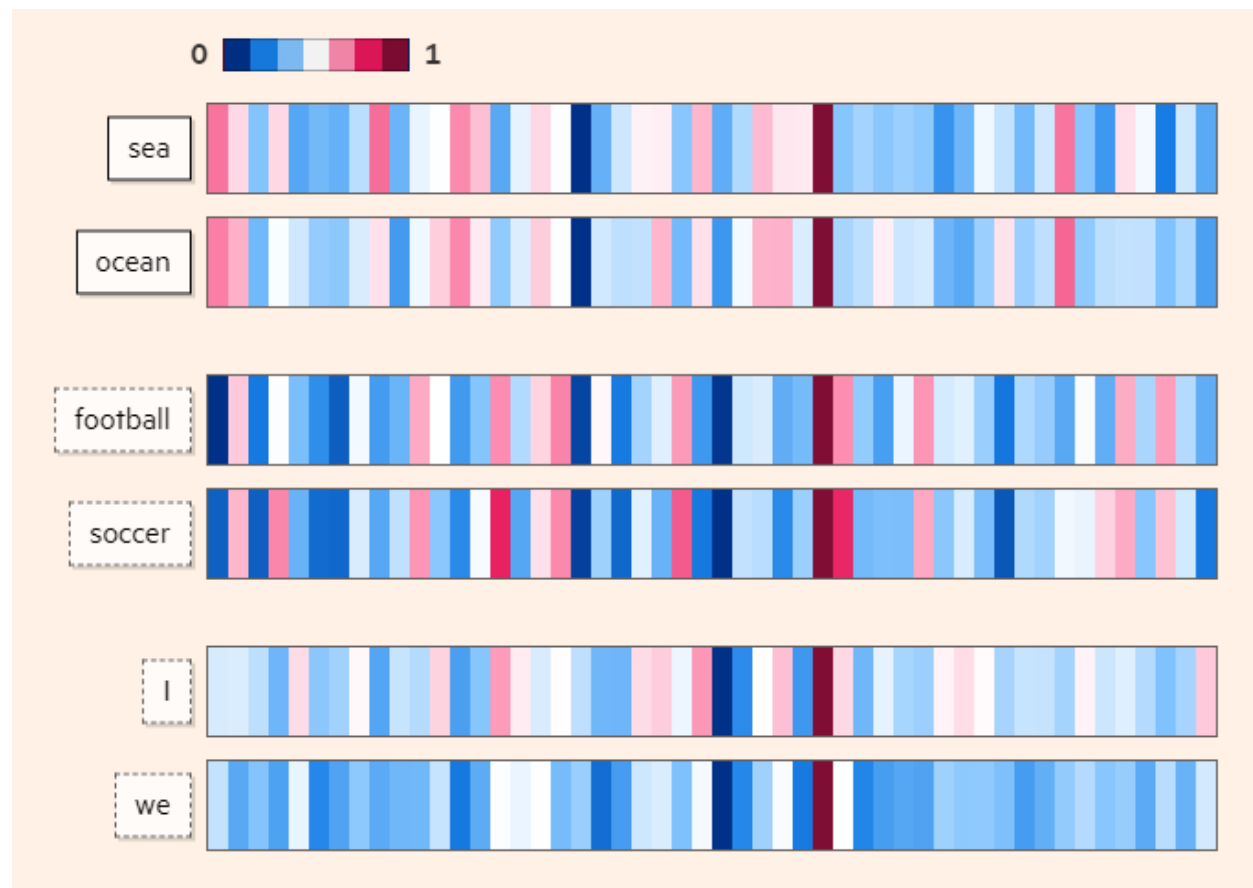
- To learn the vector embedding of a word, the model observes it in context using a large training dataset, and adjusts the vector value based on the word's proximity to other words in the training data
 - E.g., sentences with the word 'work'



Word Embeddings

6.1.2 Word Embeddings

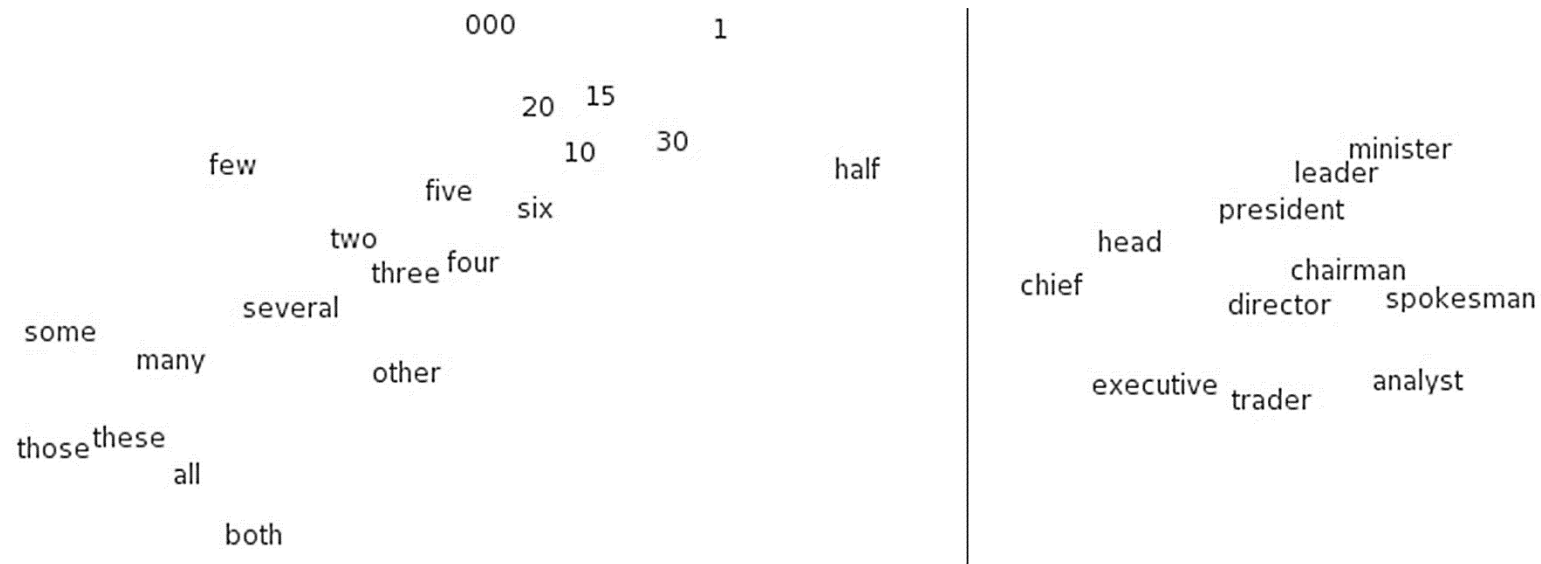
- The embedding vectors of words that have similar meanings are also similar
 - E.g., the embedding vectors of the words 'football' and 'soccer' are more similar to each other, than the embedding vectors of the words 'ocean' or 'soccer'



Word Embeddings

6.1.2 Word Embeddings

- The figure shows an example of word embeddings space
- Typically, the cosine distance between the vectors in the embedding space is used as a distance metric
 - For given embedding vectors $\mathbf{u}$ and $\mathbf{v}$ of two words, cosine similarity is $\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$



Transformer Networks

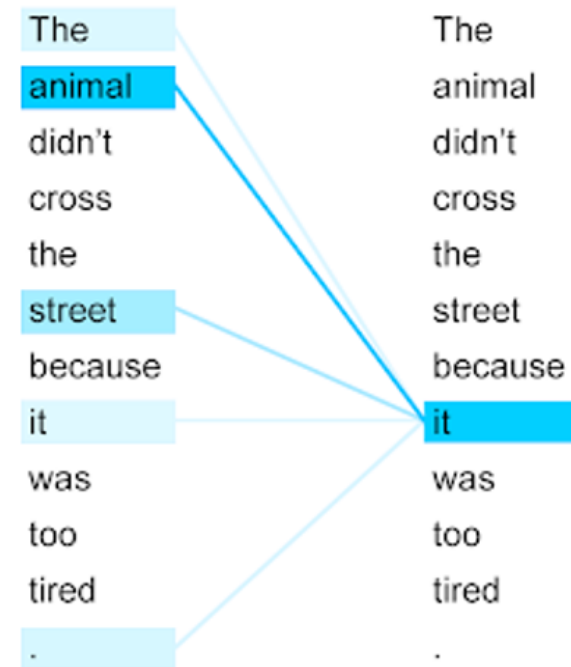
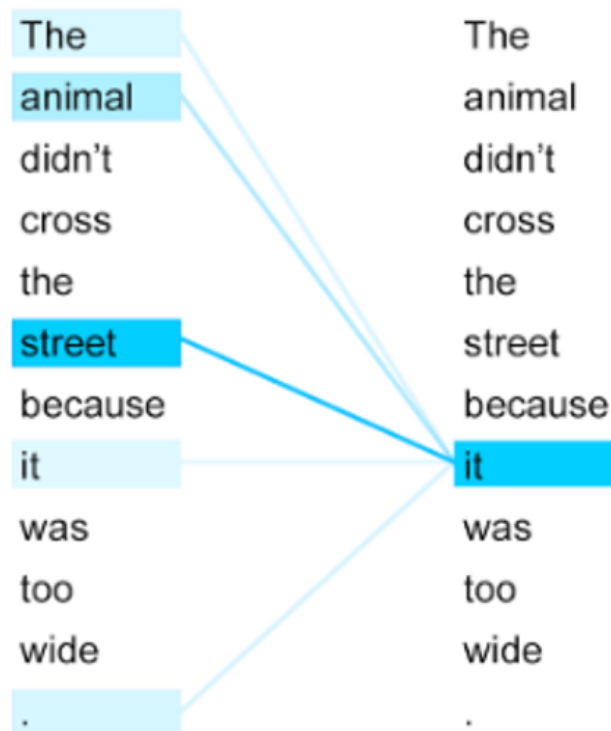
6.2 Transformer Networks

- *Transformer Neural Networks* were introduced in 2017 in the paper “Attention is all you need”
 - The title refers to the attention mechanism, which forms the basis for data processing with Transformer networks (or, simply Transformers)
- Transformers have been the main type of ML models for NLP in recent years
 - They replaced Recurrent Neural Networks in all NLP tasks
 - Almost all current LLMs employ the Transformer Networks architecture
- Beside for NLP, Transformers were also adapted for other tasks such as image processing and video processing tasks (a.k.a. Vision Transformers), protein and DNA sequence prediction, time-series data processing, etc.

Transformers: Self-attention

6.2 Transformer Networks

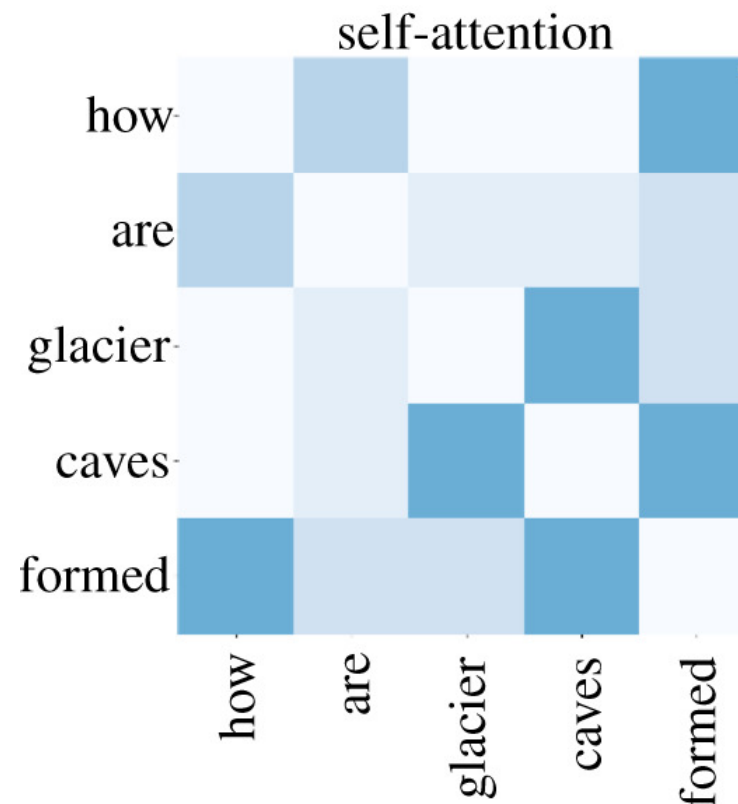
- **Self-attention** is the key layer in a Transformer network
 - It models the relationships between all words in text, and assigns weights to the words based on their importance
 - Based on context, the model should pay more attention to some words in sentences, and less attention to other words in sentences that are less relevant for the given context



Transformers: Self-attention

6.2 Transformer Networks

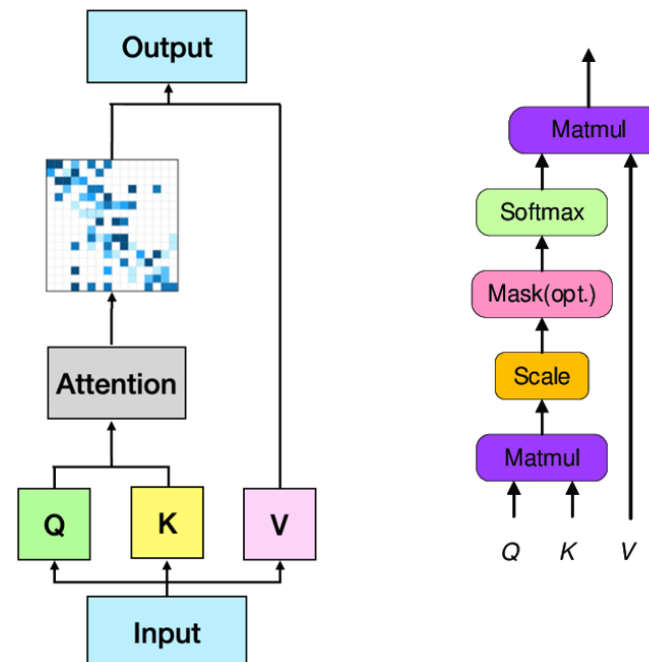
- For each **query word** Q , the attention mechanism calculates attention scores for all other words (called **key words** K)
 - The calculated *attention scores* are the dot-products $Q \cdot K$ of the input representations (embedding vectors) of the query and key words



Transformers: Self-attention

6.2 Transformer Networks

- The obtained attention scores $Q \cdot K$ for each pair of words are first scaled (dividing by the embedding dimension $\sqrt{d}$), and afterward are normalized to be in the $[0,1]$ range (by applying a softmax function)
 - I.e., the attention scores are calculated as $a_{ij} = \text{softmax}\left(\frac{Q_i K_j}{\sqrt{d}}\right)$
- Afterward, the resulted attention scores are multiplied with the initial word representation, referred to as **value** V

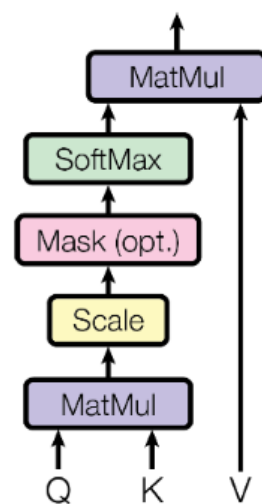


Transformers: Multi-Head Attention

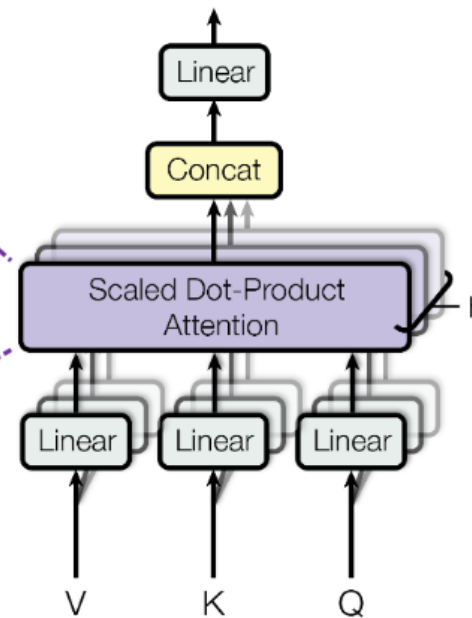
6.2 Transformer Networks

- Transformers include multiple self-attention modules, which are also referred to as attention heads
 - The combination of attention heads is called *multi-head attention*
 - Each attention head captures different relationships between the words in text
 - For example, one head may capture relationship between the adjectives in sentences, another head may focus on the relationship between the nouns and numerical values in sentences, other head may focus on rhyming words, etc.

Scaled Dot-Product Attention



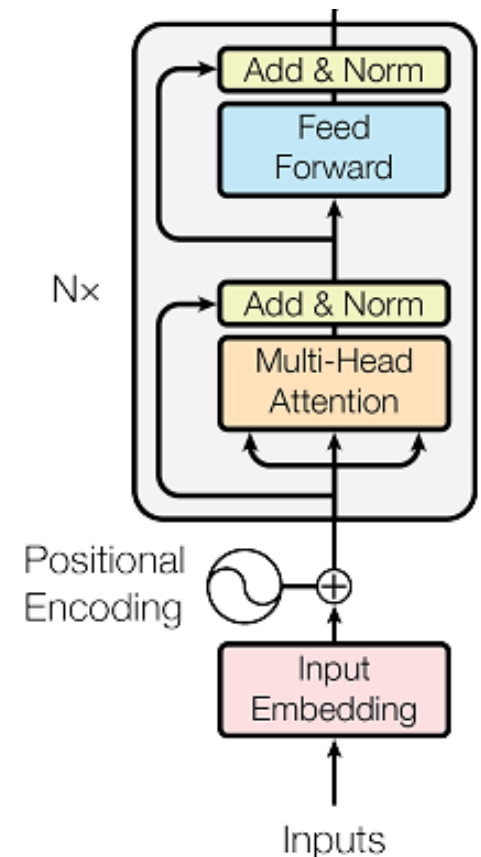
Multi-Head Attention



Transformer: Encoder

6.2 Transformer Networks

- **Encoder block** in transformers processes input text and extracts representations to be used for different NLP tasks, and it contains the following layers:
 - **Input embedding** layer
 - Takes the **embedding vector** for each word as input
 - Adds **positional encoding** vectors that provide information about the order of the words in sentences
 - **Multi-head attention** layer
 - Consisting of multiple self-attention modules
 - **Add & Norm** layer
 - **Residual connections**, add the inputs to the outputs of a layer
 - **Layer normalization**, normalizes the outputs to have 0 mean and 1 standard deviation
 - **Feed Forward** network
 - Consists of two fully-connected (dense) layers
- Larger Transformers typically include several encoder blocks in a sequence (e.g., 6 in the original paper)

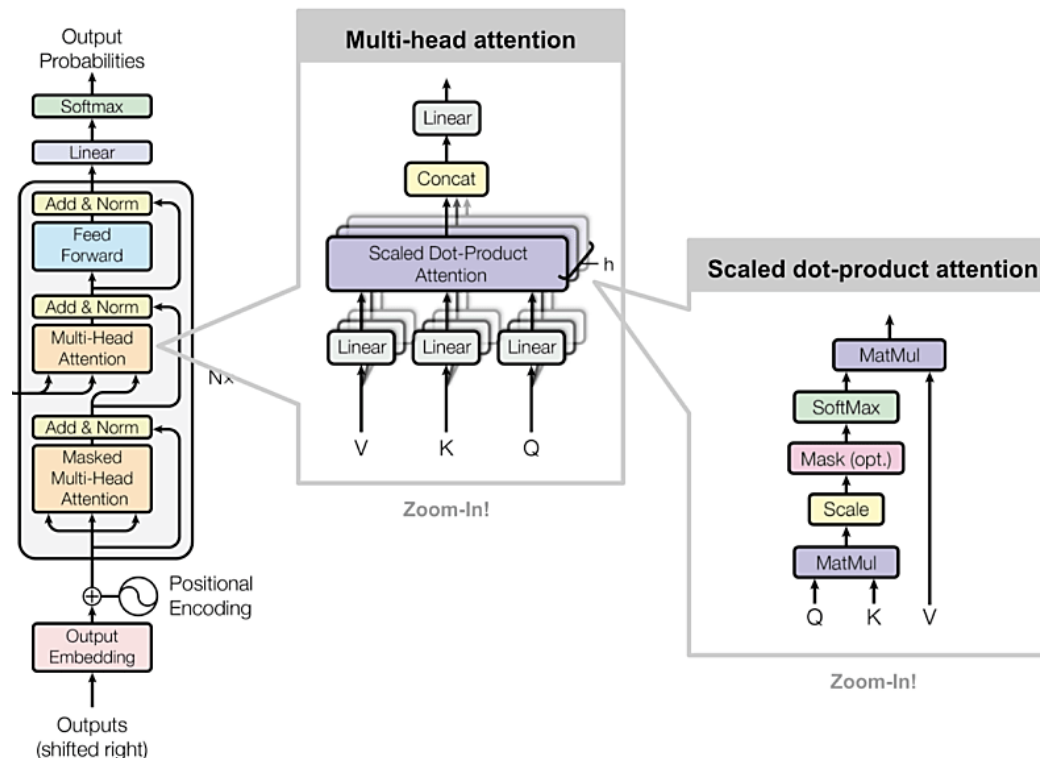


Transformers: Decoder

6.2 Transformer Networks

• Decoder block

- It is similar to the encoder block, and it is used when the output of the model is also text, such as for machine translation, text generation, and similar NLP tasks
- The main difference from the encoder is the **masked multi-head attention module**
 - This module applies masking to the next words in the text sequence, so that the network does not have access to those words during model training



Vision Transformers

6.2 Transformer Networks

- *Vision Transformers* are transformers designed for computer vision tasks
 - Input images are split into a set of smaller patches that are imputed to the model (where each image patch is considered a token)
 - The patches are flattened to 1D vectors, and processes by the network
 - Vision Transformers have outperformed Convolutional NNs on several vision tasks



Transformer Networks

6.2 Transformer Networks

- Variants of Transformer Networks are used for different tasks
 - **Decoder-only models**
 - Utilize only the decoder part of the Transformer Network architecture
 - Are particularly suitable for generating text and content
 - Majority of modern LLMs (GPT, Claude, Gemini) are decoder-only models
 - **Encoder-only models**
 - Use only the encode part of the Transformer
 - Perform well on tasks such as classification, sentiment analysis, named entity recognition
 - E.g., BERT
 - **Encoder-decoder models**
 - Employ the original Transformer Network architecture with both encoder and decoder
 - Are used for NLP tasks such as translation and summarization
 - E.g., T5 (Text-to-Text Transfer Transformer)

Large Language Models

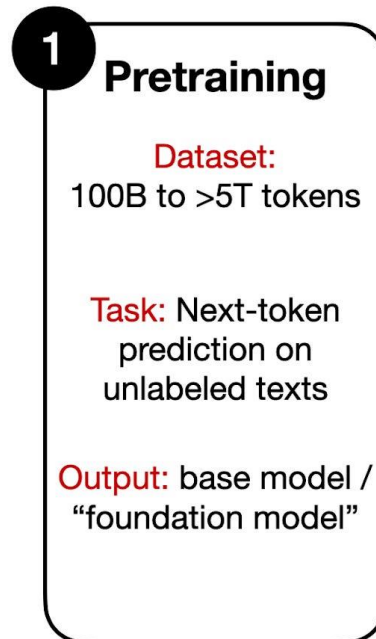
6.3 Large Language Models

- *Large Language Models* (LLMs) are advanced AI models designed for natural language understanding and generation
 - LLMs are trained on large scale across data, compute, and model size
- Creating LLMs typically involves several main phases
 - a. Pre-training**
 - The model extracts knowledge from large unlabeled text datasets
 - b. Mid-training**
 - The model is further trained on smaller, high-quality datasets containing content related to scientific knowledge, reasoning traces, coding, and long-context understanding
 - c. Post-training**
 - a. Supervised finetuning:** refined on human labeled datasets to improve responses
 - b. Alignment:** trained to generate safe and helpful responses aligned with human preferences
 - c. Reasoning, agentic finetuning:** RL training, tool use, self-reflection
 - d. Safety and red teaming:** improve robustness to adversarial prompts, bias mitigation

LLM Pre-training

6.3.1 Creating LLMs

- **Pre-training** LLMs involves processing terabytes of text (web pages, books, news articles, Wikipedia) to learn grammar, facts, and world knowledge
 - **Causal language modeling** (also known as **autoregressive language modeling**) involves training the model to predict the next token in the text sequence given the previous tokens
 - E.g., given the sentence that starts with 'Project Gutenberg ... is the oldest digital' predict the word 'library'



Project Gutenberg (PG) is a volunteer effort to digitize and archive cultural works, as well as to "encourage the creation and distribution of eBooks." It was founded in 1971 by American writer Michael S. Hart and is the oldest digital **library**. Most of the items in its collection are the full texts of books or individual stories in the public domain. All files can be accessed for free under an open format layout, available on almost any computer. As of 3 October 2015, Project Gutenberg had reached 50,000 items in its collection of free eBooks.



LLM Pre-training

6.3.1 Creating LLMs

- Given a sequence of **tokens** (word embeddings) $x_1, x_2, \dots, x_{i-1}$ from a **vocabulary** $\mathcal{V}$, the goal is to minimize the training loss in predicting the next token x_i
 - I.e., a Neural Network with parameters θ models $P_\theta(x_i|x_1, x_2, \dots, x_{i-1})$
- The model is trained to minimize the cross-entropy loss
$$\mathcal{L}(\theta) = - \sum_{i=1}^n \log f_\theta(x_i|x_1, x_2, \dots, x_{i-1})$$
 - n is the number of tokens in the vocabulary $\mathcal{V}$
- The output of the model is a probability distribution for the token x_i over the words in the vocabulary
- E.g., given the sequence “Marry had a little” based on the examples seen in the training data, the word “lamb” is the most likely next word in the sequence



LLM Pre-training

6.3.1 Creating LLMs

- Pre-training allows to extract knowledge from very large unlabeled datasets in unsupervised learning manner, without the need for manual labeling
 - Or, the “label” in pre-training is the next word in the text (e.g., ‘library’), to which we have access since it is part of the training text
 - Such learning approach is **self-supervised learning**, since the model uses each next word in the text to self-supervise the training
- Pre-training LLM is computationally expensive and time-consuming
 - It can take weeks or months on large GPU clusters, and costs millions of dollars
- The resulting LLM from the pre-training phase is called **base model**

LLM Mid-training

6.3.1 Creating LLMs

- *Mid-training* phase adapts the pre-trained base model to enhance specific capabilities
 - This stage uses next token prediction similarly to pre-training, however it employs smaller, high-quality datasets
 - The training objective is still self-supervised learning
- Mid-training typically includes:
 - **Domain adaptation** on scientific content, code datasets, medical and legal datasets
 - **Reasoning-focused training** on math problems, logical reasoning datasets
 - **Tool-use training** on structured API calls and function-calling examples
 - **Curriculum learning**, includes training that gradually progresses from easier to more difficult problems and tasks



LLM Post-Training

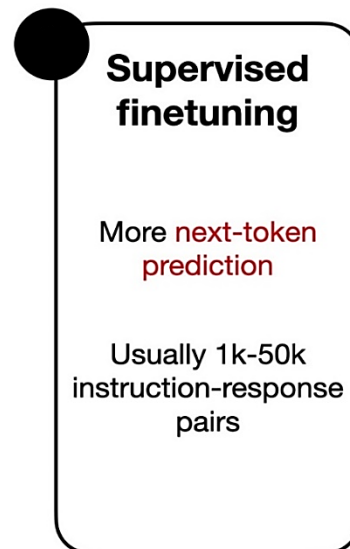
6.3.1 Creating LLMs

- The generated text by base models from pre-training and mid-training phases may not be optimized for conversations, as well as it may contain harmful or biased responses
- *Post-training phase* refines the model to optimize the response for safety and usability by end-users
- Post-training can include several model refinement stages
 - Typically, it includes supervised finetuning, alignment, reasoning finetuning, agentic finetuning, and safety and red teaming

Supervised Finetuning

6.3.1 Creating LLMs

- **Supervised finetuning** (SFT) refines the model on smaller datasets with:
 - Human-written instructions (prompts)
 - Desired responses (reference outputs)
- This training stage is also referred to as **instruction following**
- The training objective remains next-word prediction
 - The model learns to generate responses that resemble the human examples and adopt desired structure, style, and tone of the responses
 - This phase requires a laborious process of preparing dataset by human annotators



```
{  
  "instruction": "Write a limerick about a  
                pelican.",  
  "input": "",  
  "output": "There once was a pelican so fine,  
            \nHis beak was as colorful as  
            sunshine,\nHe would fish all day,\nIn  
a very unique way,\nThis pelican was  
truly divine!\n\n\n",  
},  
  
{  
  "instruction": "Identify the odd one out from  
                the group.",  
  "input": "Carrot, Apple, Banana, Grape",  
  "output": "Carrot\n\n",  
},
```

Alignment with Human Preferences

6.3.1 Creating LLMs

- *Alignment with human preferences*
 - To ensure that responses generated by LLMs are aligned with human preferences, this phase further refines the model to reduce harmful, biased, or otherwise undesirable outputs
 - Human annotators provide preferences by prompting a model to generate multiple responses and ranking the responses from best to worst
 - This stage is also known as [preference finetuning](#)
- Main strategies for LLM alignment include:
 - Reinforcement Learning from Human Feedback (RLHF)
 - Train a reward model, use Proximal Policy Optimization (PPO) to learn preferences
 - Direct Policy Optimization (DPO)
 - Eliminate the need for a separate reward model, and directly optimizes the model using preference pairs

Alignment with Human Preferences

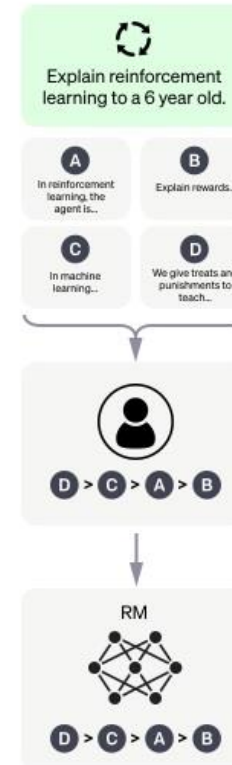
6.3.1 Creating LLMs

• Reinforcement Learning from Human Feedback (RLHF) with PPO

1. Collect human-rankings of multiple LLM responses to the same prompt, and ask human labelers to rank the responses from best to worst
2. Train a **Reward Model** to score the responses that were ranked based on human preferences
3. Apply Proximal Policy Optimization (PPO) RL algorithm to guide the LLM towards generating higher-scoring responses by maximizing a reward score

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.

This data is used to train our reward model.

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

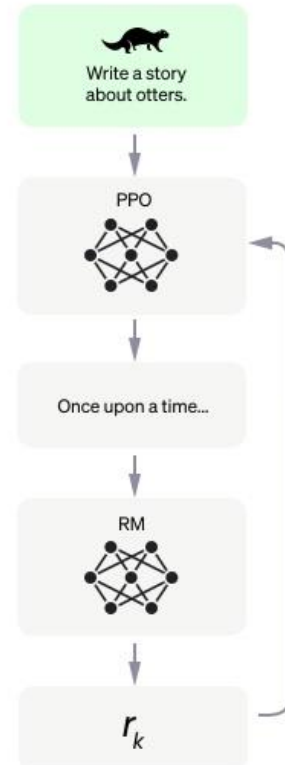
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Additional Post-training Steps

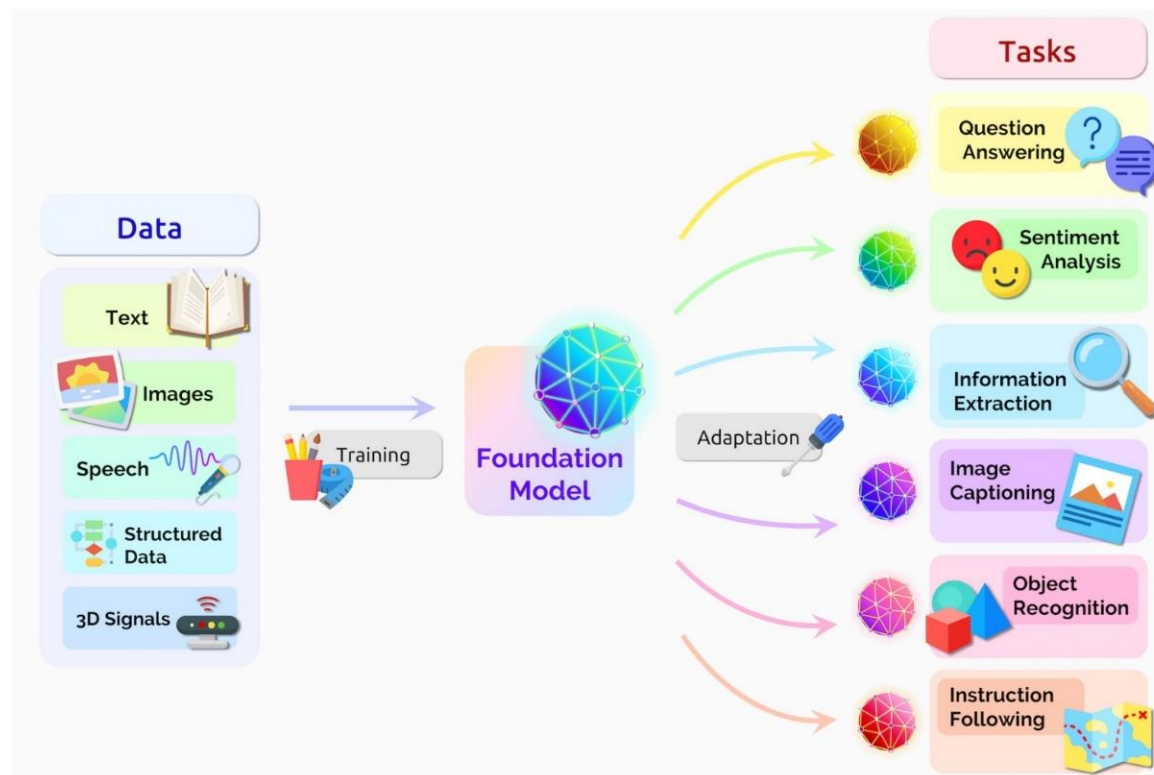
6.3.1 Creating LLMs

- Modern LLMs go through further refinement to enhance capabilities and robustness
- *Reasoning finetuning*
 - Training on multi-step problem solving, including math, logic, code
 - Typically via Reinforcement Learning with Verifiable Rewards (RLVR)
- *Agentic and tool-use training*
 - Learning how to call external tools (e.g., web search, calculator)
 - Task decomposition and planning
 - Multi-step workflow optimization
- *Safety hardening and red teaming*
 - Testing on adversarial prompts (jailbreak attempts)
 - Improving robustness to prompt injection
 - Bias and toxicity mitigation

Foundation Models

6.3.1 Creating LLMs

- *Foundation models* are scaled-up NNs across data, compute, and model size
 - Typically trained on multimodal data (text, images, audio, video, other data sources)
- The scale allows to perform transfer learning on diverse tasks (even unseen tasks) and provides for emergent capabilities
 - Foundation models have potential to replace task-specific with general models



Retrieval Augmented Generation (RAG)

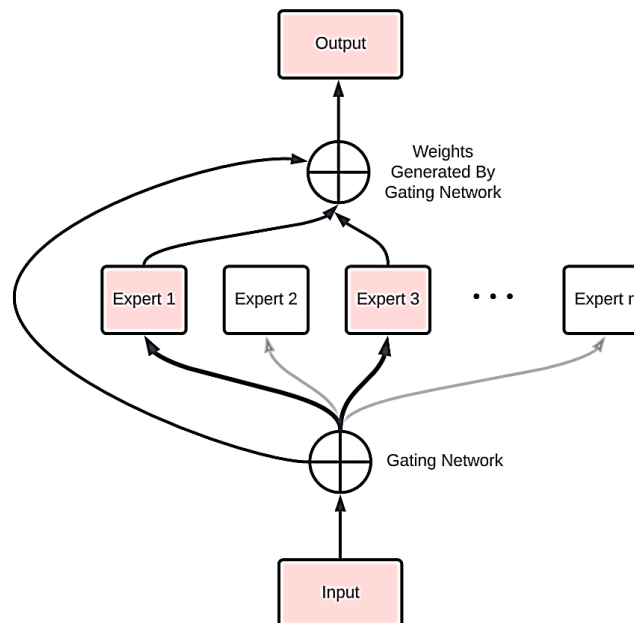
6.3.2 Retrieval Augmented Generation (RAG)

- LLM knowledge is limited to the information present in their training data (often outdated) and hallucinations when uncertain how to respond
- *Retrieval Augmented Generation (RAG)* incorporates external sources for improving LLMs outputs
 - Provides access to up-to-date information from databases, articles, web, documents
- RAG involves two phases:
 1. **Retrieval:** search for relevant information in external databases based on the user query, and identify the most relevant sources
 2. **Content generation:** utilize the retrieved information to enhance response accuracy and relevance
- Benefits of RAG
 - More accurate responses, combines LLM knowledge with current facts and up-to-date information
 - Reduced hallucinations, provides context for generation
 - Verifiable sources: users can review references used in the response

Introduction to MoE

6.3.3 Mixture of Experts

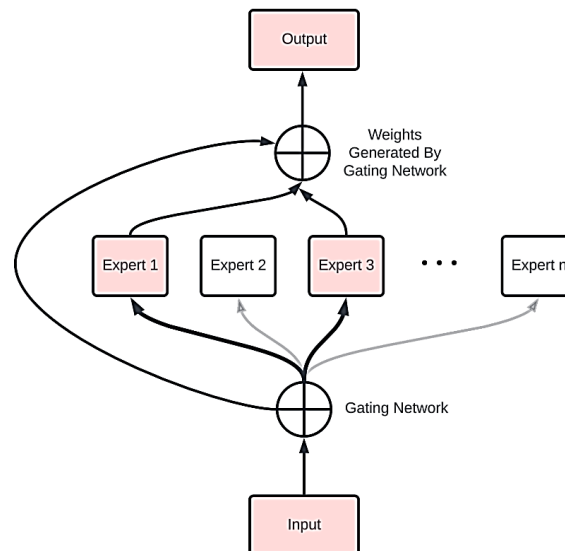
- *Mixture of Experts (MoE)* is a technique that combines multiple specialized models to handle complex tasks
 - It uses an ensemble of "expert" models focused on learning different aspects of a task
- A MoE system consists of two key components:
 - *Expert models*, which can be neural nets or network layers, trained on different parts of the inputs (tokens)
 - *Gating network*, which acts as a **router** and decides which expert(s) should process an input



Gating Network

6.3.3 Mixture of Experts

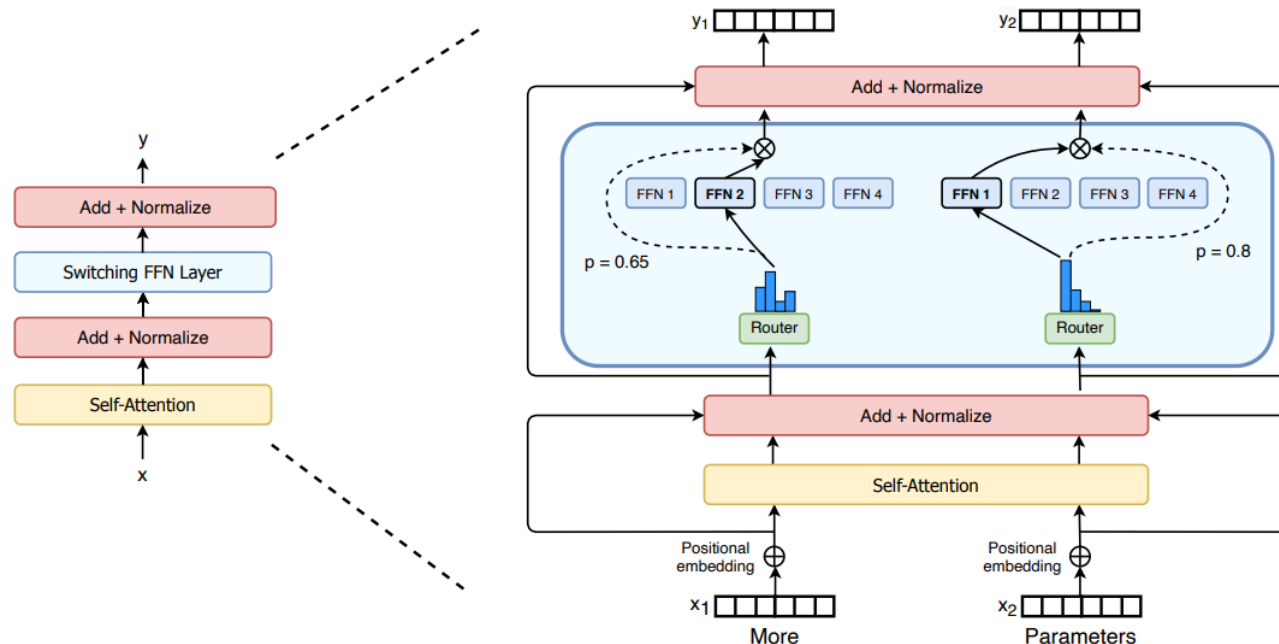
- The **gating network** analyzes the inputs, and routes each input to relevant expert(s)
 - In some MoE architectures, each token is sent to more than one expert (e.g., to 2 experts in Mixtral)
 - The experts' outputs are afterward combined through techniques like additive or multiplicative aggregation of the outputs
- During inference, for each input token only the chosen experts are activated, allowing efficient use of computational resources



Expert Models

6.3.3 Mixture of Experts

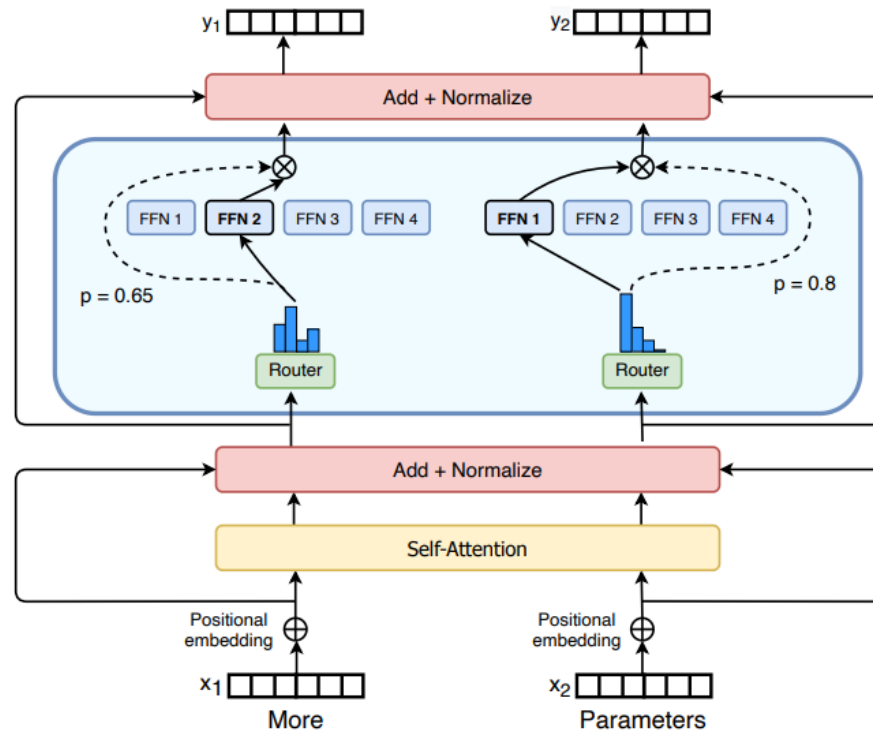
- MoE in Transformers use *sparse layers* instead of dense feed-forward network (FFN) layers
 - I.e., standard FFN layers are replaced with **switching FFN layers**
 - Each MoE layer has a certain number of experts (e.g., 4), where each expert is an FFN, and each processes specific tokens
 - For example, in the figure, the router sends the token “More” to the second expert (FFN 2), and the token “Parameters” is sent to the first expert (FFN 1)



Routing

6.3.3 Mixture of Experts

- Efficiently routing the tokens is one of the challenges for MoE
 - In recent MoE, the gating network (router) is composed of learned parameters and is trained at the same time as the rest of the network
 - I.e., the gating network is typically a simple neural network with a softmax function, which learns which expert to send each input to



Routing

6.3.3 Mixture of Experts

- In some cases, the gating network may converge to mostly activate the same few experts
 - When all tokens are sent to just 2 or 3 experts, that makes the training inefficient
 - I.e., the load is not properly balanced across all experts
- To mitigate the issue of **load balancing**, an auxiliary loss can be introduced during model training to encourage giving all experts equal importance
 - The auxiliary loss ensures that all experts receive a roughly equal number of training examples

Advantages of MoE

6.3.3 Mixture of Experts

- The key advantages of MoE include *faster pretraining* thanks to sparse layers, and *efficient inference* by activating only required experts
 - MoE LLMs are pretrained much faster than dense models with the same number of parameters
 - E.g., some LLMs based on MoE achieved a 4x pretraining speed-up over the corresponding dense models
 - The size of a model is one of the most important factors for enhanced model capacity, and given a fixed computing budget, training a larger model for fewer steps is preferred than training a smaller model for more steps
 - Faster inference is very important for end-users
 - Although an MoE might have many parameters, only some of them are used during inference, which results in much faster inference compared to a dense model with the same number of parameters
- Similarly, MoE generate *higher quality outputs* by combining specialized experts

Challenges of MoE

6.3.3 Mixture of Experts

- *Finetuning* MoE *is more challenging* than dense models, because sparse layers are more prone to overfitting
 - Recent works have shown promising results and introduced strategies for finetuning MoE
- MoE still require all experts to be loaded into the memory for both training and inference
- In addition, MoE require to carefully balance tradeoffs between training costs and inference efficiency
 - While offering faster inference, MoE require careful management of VRAM and computing resources



Limitations of LLMs

6.4 Limitations of LLMs

- Computational resources
 - High requirements for training and access
- Data bias
 - Reflecting potential biases present in training data
- Hallucinations
 - Generating false or inappropriate information
- Inexplicability/black-box nature
 - Lack of transparency in reasoning and decision-making



Ethical Considerations

6.4 Limitations of LLMs

- Privacy risks with sensitive information in training data
- Misinformation and manipulation potential
- Access and fairness limitations
- Environmental impact of energy consumption



Responsible LLM Development

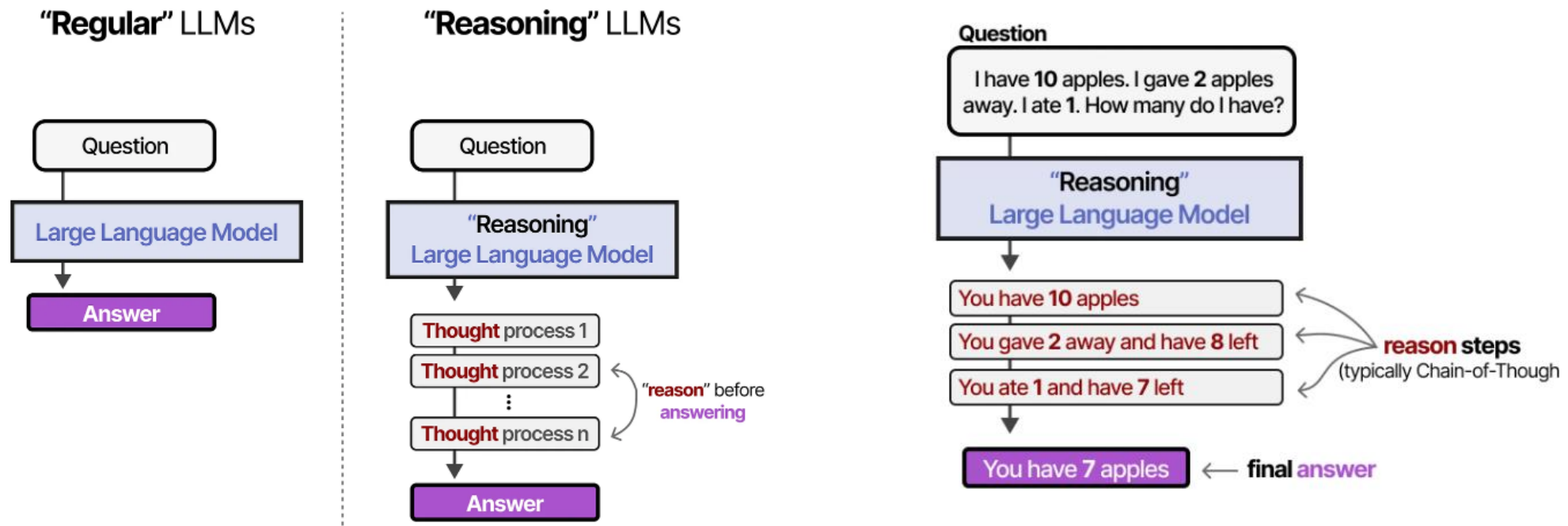
6.4 Limitations of LLMs

- Responsible LLM development requires:
 - Transparency and collaboration in research and development
 - Mitigating bias and ensuring fair representation in data
 - Addressing misinformation and promoting responsible use
 - Open access and democratization of AI benefits
 - Sustainable practices for environmental impact

Reasoning Models

6.5 Reasoning Models

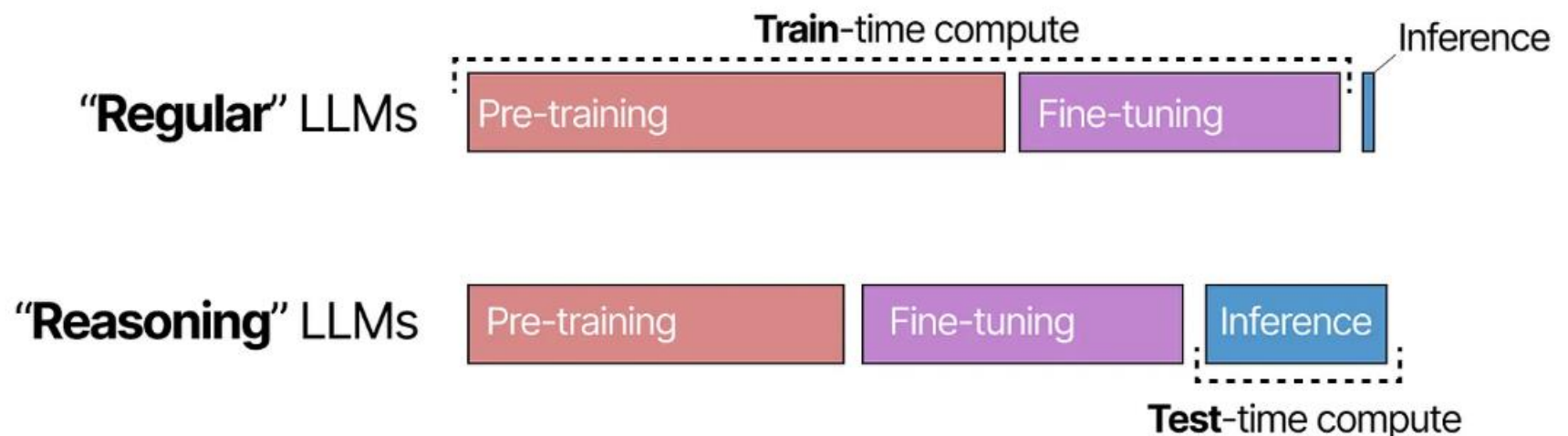
- **Reasoning LLMs** perform multiple reasoning steps or thought process before providing an answer
 - The **reasoning steps** or **thought process** break down the inference process into smaller, structured inferences (or **Chain-of-Thought**), instead of answering directly
 - LLM developers describe the reasoning process as the model spending more time 'thinking' through the problem before responding



Reasoning Models

6.5 Reasoning Models

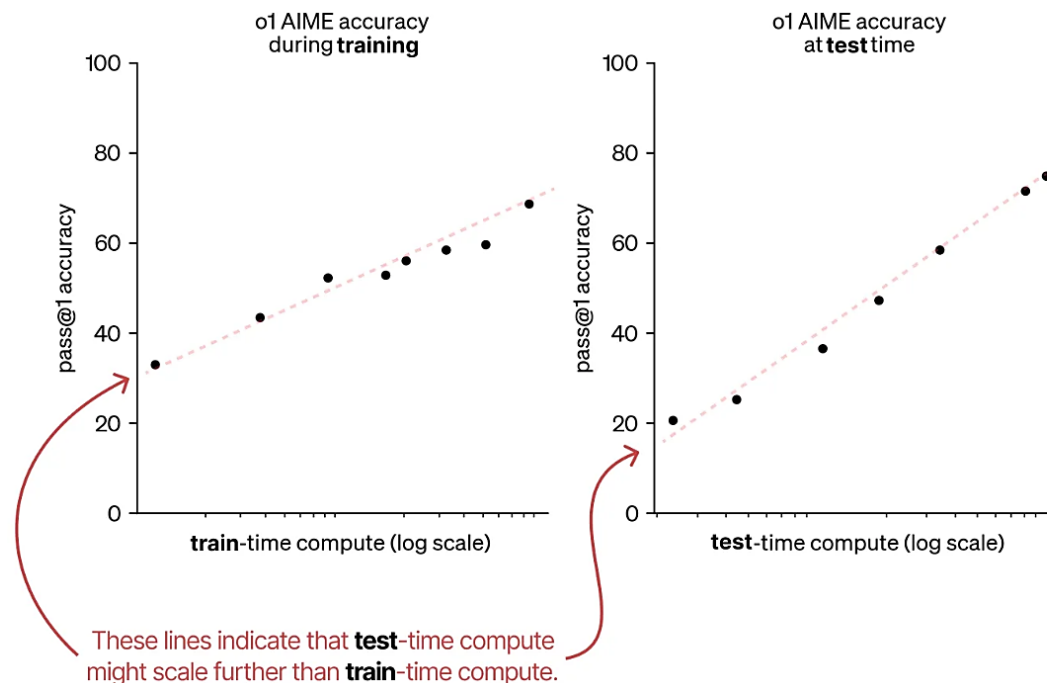
- Until 2024, to increase the performance of LLMs during pre-training, developers scaled the *train-time compute* by increasing:
 - Dataset size (number of tokens)
 - Model size (number of parameters)
 - Computational resources (number of GPUs)
- Instead of increasing pre-training budgets, *test-time compute* allows LLMs to “think longer” during inference



Reasoning LLMs

6.5 Reasoning Models

- LLM *scaling laws* provide a relationship between the model's performance and model's scale (typically shown on a log scale)
 - Well known **train-time compute scaling laws** include Chinchilla and Kaplan laws
 - They indicate that performance increases with more compute, tokens, and parameters
- OpenAI's **test-time compute scaling law** suggests that test-time compute follows the same trend as train-time compute



Categories of Reasoning Models

6.5 Reasoning Models

- *Inference-based reasoning methods*
 - Rely on prompting and sampling techniques that improve reasoning without modifying model weights
 - The model is encouraged to reason through intermediate steps before producing the final answer
 - Complex problems are decomposed into simpler subproblems that are easier to solve
- Methods
 - **Chain-of-Thought (CoT) Prompting**
 - Appending a phrase such as "Explain your response step by step" to the prompt
 - The model produces intermediate steps before the final answer, which helps generate the correct answer
 - **Self-Consistency Sampling**
 - Generates multiple responses and select the most frequent answer as the final answer
- Limitations
 - Do not provide the model with new knowledge, only change how existing knowledge is used

Supervised Finetuning Reasoning Models

6.5 Reasoning Models

- *Supervised finetuning (SFT) reasoning models*
 - Developed by finetuning a pre-trained model on a reasoning dataset
 - The dataset provides long responses with CoT reasoning steps for every question
 - The finetuning process is the same as instruction finetuning
- Advantages
 - Model weights are updated during finetuning, which usually improves reasoning capabilities over inference-based approaches
 - SFT is straightforward to implement
- Limitations
 - Requires large, carefully curated datasets of reasoning traces
 - Models tend to memorize the reasoning in the finetuning examples rather than learning the underlying logic
 - Performance can degrade on unseen problems not represented in the dataset

RL-based Reasoning Models

6.5 Reasoning Models

- *Reinforcement Learning-based reasoning models*
 - Use reward signals to evaluate the quality of intermediate reasoning steps or final answers
 - RL models are applied to **verifiable tasks** where the correctness of an answer can be automatically checked: hence it is referred to as **RLVR (Reinforcement Learning with Verifiable Rewards)**
 - Examples: math problems (compare answer to ground truth), programming tasks (run code against test cases)
 - Unlike SFT reasoning models which minimize cross-entropy loss, RL-based models are finetuned to maximize a reward signal
 - This allows the model to learn logic and principles helpful for solving new, unseen problems
 - RL reasoning models often outperform inference-based or SFT models on complex tasks requiring mathematical or logical reasoning
- Limitations
 - More difficult to finetune than SFT
 - Risk of reward hacking: the model finds shortcuts to maximize rewards but produces incorrect CoT reasoning

Reasoning LLMs

6.5 Reasoning Models

- When do we need reasoning LLMs
 - E.g., factual questions like “What is the capital of USA?” do not need reasoning
 - E.g., the question “If a train is moving at 60 mph and travels for 3 hours, how far does it go?” requires some level of reasoning
- Strengths and limitations of reasoning LLMs

Good at	Bad at
+ Deductive or inductive reasoning (e.g., riddles, math proofs)	– Fast and cheap responses (more inference time)
+ Chain-of-thought reasoning (breaking down multi-step problems)	– Knowledge-based tasks (hallucination)
+ Complex decision-making tasks	– Simple tasks (“overthinking”)
+ Better generalization to novel problems	

Agentic AI

6.6 Agentic AI

- *AI Agents* use LLMs to reason through tasks and take actions using digital tools or act in environments
- Abilities
 - Plan multi-step solutions
 - Call external tools (e.g., web search, code execution)
 - Adapt behavior based on feedback
- Examples of use
 - Automating document analysis
 - Managing workflows (e.g., email sorting, report writing)
 - Scientific discovery and literature review



Agentic AI

6.6 Agentic AI

- An agent is not simply an LLM that outputs text: it is a computational entity capable of reasoning, planning, and executing actions to achieve a goal
- Unlike a standard LLM (single pass to produce an output), an agent operates within a **continuous loop**
 - The loop involves: observing the environment, reasoning about the current state, formulating a plan, executing actions, and using the output of those actions to refine the next steps
- Example
 - Task "Plan a marketing campaign for next quarter"
 - The agent breaks the task into smaller steps, use external tools like web search or databases, check the results, and iterate until complete

Multi-Agent Systems

6.6 Agentic AI

- Multiple autonomous agents work collaboratively to achieve a goal
 - Each agent has its own specialized tools, reasoning abilities, or knowledge sources
- Complex tasks are divided by expertise
 - E.g., one agent for web search, another for math computations, a third for summarization
- Example: Manager + Specialist pattern
 - Manager plans the workflow and integrates results
 - Web Specialist runs tools and reports back to the Manager

Challenges and Risks in Agentic AI

6.6 Agentic AI

- Safety, security, and system access
 - Agents can execute code, issue transactions, modify files, and interact with APIs , which greatly expand failure consequences
 - A single hallucination in healthcare or finance can cause harm
 - Adversarial prompts and malicious tool responses are serious threats
 - Secure authentication and strict permissions are critical for safe operation
- Ethical, legal, and societal questions
 - Accountability: developers, deployers, or users — who is responsible when an agent makes a mistake?
 - Transparency, fairness (preventing bias amplification), and privacy of confidential data are key concerns